Software Engineering (sessional) [CSE-356]

Proposal Report on:

Niramoy: An AI-Assisted Serverless Telemedicine & Appointment Platform

Group No:

Name	**Student ID**
Fariha Rayhan Mim	2204104
Dipta Deb Nath	2204103

Submitted to:
Mir Md. Saki Kawsar
Assistant Professor,
Department of CSE, CUET

Md. Refaj Hossain
Lecturer,
Department of CSE, CUET

Department of Computer Science and Engineering (CSE)
Chittagong University of Engineering & Technology (CUET)
Chattogram-4349, Bangladesh

1 Introduction / Background

Access to qualified doctors in Bangladesh is heavily concentrated in a few urban centres, while patients in rural and semi-urban areas face long travel, long queues, and no reliable way to reach specialists. Appointment booking is still largely manual — phone calls, chamber visits, and middlemen — which causes double-booking, no-shows, lost medical history, and no price or quality transparency. There is no centralized, trustworthy platform where a patient can discover a verified doctor, book a conflict-free slot, consult over video, and keep a permanent digital record of prescriptions and history.

Niramoy addresses this gap: a web platform connecting patients and doctors for remote consultation, built around a scheduling engine that guarantees conflict-free booking, an embedded video call, digital prescriptions, a persistent medical history, and — as its differentiator — an AI layer that triages symptoms, recommends the right doctor, and auto-summarizes each consultation. The platform is designed to be lightweight, low-cost, and deployable entirely on free-tier serverless infrastructure, making it realistic to build and host within a course timeline while still solving a genuine, high-impact problem.

2 Project Objectives

Main Objective: To develop a reliable, role-based telemedicine platform that lets patients book conflict-free appointments with verified doctors, consult via video, and manage digital prescriptions and medical history, augmented by AI for triage, recommendation, and record-keeping.

Specific Objectives

- Enable patients to register, search/filter verified doctors, and book/cancel appointment slots with guaranteed no double-booking (database-level concurrency control).
- Let doctors define weekly availability with exceptions, run consultations, and issue digital prescriptions.
- Provide an AI health-assistant chatbot plus an AI symptom-triage assistant that estimates urgency and suggests the correct specialty.
- Provide AI doctor matching and slot recommendation based on the patient's described problem.
- Generate AI visit summaries and prescription drafts, and maintain a secure, role-gated medical-history timeline.

3 Scope of the Project

What Will Be Included

- Doctor availability rules with exceptions, auto-generated bookable slots.
- Conflict-free appointment booking, rescheduling, and cancellation, stored in UTC for timezone safety.

- Waitlist with automatic notification when a booked slot frees up.
- Embedded video consultation for each appointment.
- Digital prescriptions with PDF export and a persistent patient medical-history timeline.
- "My Appointments" dashboard and a doctor earnings dashboard.
- Doctor search with filters for specialty, fee, rating, language, and availability.
- In-app notification bell, scheduled reminders, and automatic no-show marking.
- AI features: symptom triage, doctor matcher/recommendation, and visit summary.
- Admin panel to verify doctors and manage specialties.

What Will Not Be Included

- Real payment gateway integration (a mock flow is used for the MVP).
- Insurance claims processing and pharmacy delivery integration.
- Lab-report integration and a native mobile application.
- Formal regulatory (HIPAA-style) certification, which is out of scope for a course project.

4 Proposed Features

Core Features

- **Authentication & Roles** — Patient / Doctor / Admin, with role-gated routes.
- **Doctor Availability Engine** — recurring rules, exceptions, buffer time, and automatic slot generation.
- **Conflict-Free Booking** — a unique constraint on doctor and start time enforced inside a transaction, so double-booking is impossible; a cancellation window is enforced.
- **Video Consultation** — one video room generated per appointment.
- **Digital Prescriptions & Medical History** — structured prescription items, stored persistently, and shared only with role-gated access.
- **Notifications & Reminders** — scheduled "appointment in 1 hour" reminders and automatic no-show marking.
- **Reviews & Ratings** — patients rate completed consultations.

Additional Features

- Prescription PDF export, generated from the prescription form and stored in cloud blob storage.
- "My Appointments" dashboard with upcoming, past, and cancelled tabs, tailored per role.

- Doctor search filters by specialty, fee range, rating, language, and availability.
- In-app notification bell for real-time reminders and status changes.
- Waitlist with auto-rebooking notification for the next patient in line.
- Family accounts to book and manage appointments on behalf of a child or parent.
- Consultation notes timeline, a chronological medical-history view across all visits.
- Doctor earnings dashboard summarizing completed consultations and trends.
- Rescheduling flow that re-runs the conflict-free check on a new slot.

AI Integration (Mandatory — multiple features)

- **AI Health Assistant Chatbot** — answers general health questions, explains prescriptions and medical terms, and guides users through the app, with a permanent safety disclaimer since it is not a diagnostic tool.
- **AI Symptom Triage Assistant** — reads free-text symptoms and returns an urgency level (self-care / see doctor / urgent) along with the recommended specialty.
- **AI Doctor Matcher & Slot Recommendation** — recommends the best-matching verified doctors and convenient available slots from the patient's described problem.
- **AI Visit Summary** — summarizes the consultation and drafts a structured prescription for the doctor to review and confirm.

5 Methodology / Approach

Chosen SDLC Model: Incremental — each module (authentication, scheduling, booking, video, prescriptions, and AI) is built and tested in increments, which allows early working demos and lets AI integration land later in the timeline without blocking the core system.

Technology Stack

- **Frontend:** Next.js (React, App Router), Tailwind CSS.
- **Backend:** Next.js Route Handlers and Server Actions (serverless).
- **Database:** Vercel Postgres (Neon), using unique constraints and transactions for booking safety.
- **Storage:** Vercel Blob, for prescription PDFs and records.
- **Auth:** NextAuth (Auth.js), with the user's role embedded in the session.
- **Scheduling:** Vercel Cron, for reminders and the no-show sweep.
- **Video:** Jitsi or Daily.co, embedded via iframe.
- **Email:** Resend, for transactional notifications.

AI Technology: An LLM API (Gemma or OpenAI) handles triage, recommendation ranking, and summarization, orchestrated through LangChain; a lightweight scikit-learn model is an optional addition for demand and no-show prediction.

6 Expected Outcomes / Deliverables

- A deployed, working web application (public URL) covering the full MVP.
- Key AI features delivered end-to-end: health-assistant chatbot, symptom-triage assistant, doctor matcher/recommendation, and AI visit summary.
- A conflict-free scheduling engine verified against double-booking and boundary-slot test cases.
- Full SDLC documentation: proposal, project-management plan, software requirements specification, architecture and design (ER diagram, state machine, sequence, use-case, and data-flow diagrams).
- A requirements-based automated test suite covering double-booking, past-date, boundary-slot, and authentication/authorization cases.

7 Tools & Resources Required

Hardware: A laptop with at least 8 GB RAM and a stable internet connection for both team members.

Software / APIs / Cloud Services: Vercel (hosting, Postgres, Blob, and Cron), Next.js, NextAuth, the Jitsi or Daily video API, Resend for email, the Gemma or OpenAI API for the AI layer, VS Code, GitHub for version control, Figma for UI design, and Postman for API testing.

8 Conclusion

Niramoy tackles a genuine, high-impact problem — equitable access to doctors in Bangladesh — with a design whose engineering core, a conflict-free scheduling engine, role-based access, and a consultation state machine, is well suited to demonstrating sound software-engineering practice. Its multi-feature AI layer, combining a chatbot, symptom triage, doctor recommendation, and automated visit summaries, turns a standard booking system into an intelligent, patient-centric healthcare assistant. Built entirely on free, serverless infrastructure, Niramoy is both an achievable course project and a socially meaningful platform with real potential beyond the classroom.